Labo 06 — Données et volumes

Théorie — pourquoi tout ce qu'un conteneur écrit disparaît avec lui, les trois façons de lui donner un disque qui survit, et ce que le mode rootless change aux permissions.

Objectifs

- Comprendre pourquoi la couche *writable* d'un conteneur n'est pas un lieu de stockage.
- Distinguer **volume nommé**, **bind mount** et **tmpfs**, et savoir lequel choisir.
- Lire et écrire `-v` et `--mount`, y compris `:ro`, `:U` et `:Z`.
- Expliquer qui possède les fichiers d'un volume en rootless, et pourquoi `ls` vous dit « Permission denied ».
- Sauvegarder et restaurer les données d'une base PostgreSQL conteneurisée, de deux manières.

---

1. La couche writable est un brouillon

Au labo 02, vous avez vu qu'une image est une pile de couches en lecture seule. Quand un conteneur démarre, le moteur ajoute par-dessus une couche **writable** (inscriptible), vide, propre à ce conteneur. Tout ce que le processus écrit — fichiers créés, fichiers modifiés, fichiers « supprimés » — atterrit dans cette couche. `podman diff conteneur` la liste.

> **→ LINUX**
> C'est **overlayfs**, un système de fichiers du noyau, qui empile ces couches. Il présente une vue unifiée de plusieurs répertoires : les *lowerdirs* (les couches de l'image) et un *upperdir* (la couche writable). Écrire un fichier le crée dans l'upperdir ; modifier un fichier d'une couche inférieure le **copie** d'abord vers le haut (*copy-up*), puis le modifie là ; le supprimer dépose un *whiteout*, un marqueur qui le masque. L'image n'est jamais touchée.

Cette couche a trois défauts pour des données qui comptent. Elle **meurt avec le conteneur** : `podman rm` l'efface, et un conteneur se remplace plus souvent qu'on ne croit (nouvelle image, nouveau paramètre, redémarrage par un orchestrateur). Elle est **lente** pour les écritures intensives, à cause du copy-up et de l'indirection overlay. Et elle est **invisible** des autres conteneurs : impossible de la partager.

> **À RETENIR**
> Un conteneur est **jetable** par conception. Si une donnée doit survivre au conteneur, elle ne doit pas être *dans* le conteneur. C'est le principe qui gouverne tout ce labo — et, plus tard, la séparation entre Pods et PersistentVolumes dans Kubernetes.

2. Trois façons de monter un disque

Le moteur sait **monter** un emplacement extérieur à un chemin du conteneur. Ce qui est écrit à ce chemin ne passe pas par la couche writable. Trois types existent :

Type	Où vivent les données	Qui gère	Quand l'utiliser
Volume nommé	Dans le stockage du moteur (`~/.local/share/containers/storage/volumes/`)	Le moteur : `podman volume …`	Données d'une base, d'un cache, d'une file : tout ce qui doit survivre sans que vous ayez à savoir *où*
Bind mount	N'importe quel répertoire ou fichier de l'hôte, choisi par vous	Vous	Code source en développement, fichier de configuration, dossier de sortie
tmpfs	En mémoire vive, jamais sur disque	Le noyau	Fichiers temporaires, secrets déchiffrés, caches jetables

Deux syntaxes les décrivent. La courte, `-v source:destination[:options]`, est la plus répandue ; la longue, `--mount type=…,source=…,destination=…`, est plus lisible et refuse silencieusement moins d'erreurs :

```
podman run -v pgdata:/var/lib/postgresql/data …                    # volume nommé
podman run -v ./html:/usr/share/nginx/html:ro …                    # bind mount, lecture seule
podman run --tmpfs /scratch:size=64m …                             # tmpfs
podman run --mount type=volume,source=pgdata,destination=/var/lib/postgresql/data …
podman run --mount type=bind,source=./html,destination=/usr/share/nginx/html,ro …
```

La règle qui les départage : une source qui commence par `/`, `./` ou `~` est un chemin hôte, donc un **bind mount** ; un simple nom est un **volume**. Oubliez le `./` devant `html` et vous créez un volume nommé `html`, vide — l'une des erreurs les plus fréquentes.

3. Les volumes nommés

Un volume nommé est un répertoire que le moteur crée, nomme, et conserve tant que vous ne le supprimez pas. `podman volume create pgdata` le crée ; `podman run -v pgdata:…` le monte (et le crée s'il n'existe pas) ; `podman volume inspect pgdata` révèle son `Mountpoint` sur l'hôte ; `podman volume rm pgdata` le détruit — refusé tant qu'un conteneur l'utilise.

Au **premier** montage sur un répertoire non vide de l'image, le moteur **copie** le contenu de l'image dans le volume vide (`NeedsCopyUp: true` dans l'inspect). C'est ce qui permet de monter un volume sur `/etc/nginx` sans obtenir un répertoire vide. Aux montages suivants, il ne copie plus rien.

> **PIÈGE**
> Beaucoup d'images déclarent `VOLUME /var/lib/postgresql/data` dans leur Dockerfile. Si vous lancez `postgres` **sans** `-v`, le moteur crée quand même un volume, **anonyme**, nommé par un hash de 64 caractères. `podman rm` du conteneur ne le supprime pas (`rm -v` le fait). Après quelques semaines, `podman volume ls` liste des dizaines de volumes orphelins et personne ne sait lequel contient quoi. Nommez toujours vos volumes.

→ POSTGRESQL
L'image officielle lance `initdb` au démarrage **si et seulement si** le répertoire `PGDATA` (`/var/lib/postgresql/data`) est vide : il crée alors le cluster, le superutilisateur `postgres` avec le mot de passe `POSTGRES_PASSWORD`, et exécute les scripts de `/docker-entrypoint-initdb.d/`. Si le répertoire contient déjà des données, il les réutilise sans rien initialiser — et ignore `POSTGRES_PASSWORD`. Un volume persistant change donc le comportement du conteneur au second démarrage.

4. Les bind mounts

Un bind mount rend un chemin de l'hôte visible dans le conteneur, **sans copie** : c'est le même répertoire, vu de deux endroits. Modifier `html/index.html` sur l'hôte change immédiatement ce que nginx sert ; écrire depuis le conteneur crée un fichier dans votre dossier. C'est l'outil du développement (recharger sans rebuild) et de la configuration (`-v ./nginx.conf:/etc/nginx/nginx.conf:ro`).

L'option `:ro` interdit l'écriture depuis le conteneur : une tentative échoue avec `Read-only file system`. Mettez-la par défaut, et retirez-la seulement quand le conteneur doit écrire.

→ SÉCURITÉ
Sur Red Hat, Fedora et CentOS, **SELinux** étiquette chaque fichier et interdit à un conteneur de lire ce qui ne porte pas l'étiquette `container_file_t`. Un bind mount y échoue avec `Permission denied` même en root, tant qu'on n'ajoute pas `:z` (partagé entre conteneurs) ou `:Z` (privé à ce conteneur), qui ré-étiquettent le répertoire. Ubuntu (donc WSL) utilise AppArmor, pas SELinux : `:Z` y est accepté et sans effet. Prenez l'habitude de l'écrire : vos commandes marcheront aussi sur les serveurs RHEL de l'entreprise.

→ WINDOWS / WSL
Vos fichiers Windows sont visibles sous `/mnt/c/…`, mais ce montage (`drvfs`, protocole 9p) est **lent** et ne gère pas les permissions Linux : un bind mount depuis `/mnt/c` fonctionne, puis surprend par ses `chmod` sans effet et ses temps d'accès. Gardez les fichiers des labos dans le système de fichiers Linux (`~/labo-docker`), et ouvrez-les depuis Windows via `\\wsl$\Ubuntu\home\…` si besoin.

5. Rootless : qui possède les fichiers ?

Au labo 01, vous avez vu que le `root` d'un conteneur rootless est votre propre utilisateur. Le reste de la table suit : l'UID 1 du conteneur correspond au premier UID de votre plage `/etc/subuid`, l'UID 2 au second, etc.

→ LINUX
Un **user namespace** donne à un processus sa propre table d'UID. Le noyau traduit chaque UID du conteneur en un UID réel de l'hôte selon `/proc/<pid>/uid_map`, trois colonnes : *début conteneur*, *début hôte*, *longueur*. Sur une Ubuntu/WSL standard, `podman unshare cat /proc/self/uid_map` affiche `0 1000 1` (votre UID devient root) puis `1 100000 65536` (le reste de la table pointe dans la plage subuid). L'utilisateur `postgres` de l'image, UID 70, écrit donc ses fichiers avec l'UID hôte **100069**.

Conséquence très concrète : les fichiers du volume PostgreSQL appartiennent, vus de l'hôte, à l'UID 100069, et votre `ls` sur le `Mountpoint` répond `Permission denied`. Ce n'est pas une panne, c'est l'isolation qui fonctionne. Pour y accéder, entrez dans le namespace : `podman unshare ls -ln …` affiche alors `70 70`, comme dans le conteneur.

Le problème inverse frappe les bind mounts : votre dossier appartient à l'UID 1000, que le conteneur voit comme **root**. Un processus qui tourne en `--user 1000` dans le conteneur (UID hôte

100999) n'a **aucun droit** dessus. Trois remèdes, du plus simple au plus propre :

Remède	Effet	Quand
`:U` sur le montage	Le moteur fait un `chown` récursif vers l'UID du conteneur	Dossier dédié au conteneur
`--userns=keep-id`	Votre UID garde le même numéro dans le conteneur	Outils de dev qui écrivent dans vos sources
`podman unshare chown`	Vous changez le propriétaire « vu du conteneur »	Réparer après coup

> **PODMAN**
> Docker en mode classique n'a pas ce problème : le daemon est root, et un fichier écrit par l'UID 70 du conteneur appartient *vraiment* à l'UID 70 de l'hôte — ce qui est un autre problème, de sécurité celui-là. `podman unshare`, `:U` et `--userns=keep-id` sont propres à Podman. Et `podman volume export` / `podman volume import`, qui archivent un volume en tar sans conteneur, n'existent pas dans Docker.

6. tmpfs : un disque en mémoire

`--tmpfs /scratch:size=64m` monte un système de fichiers en RAM : rapide, limité en taille, effacé à l'arrêt du conteneur, jamais écrit sur le disque de l'hôte. C'est l'endroit pour un fichier temporaire volumineux, ou pour un secret déchiffré qui ne doit laisser aucune trace. Attention au compteur : la mémoire d'un tmpfs compte dans la limite `--memory` du conteneur (labo 10).

7. Sauvegarder et restaurer

Deux familles, qui ne se remplacent pas :

- **Sauvegarde logique** : on demande au logiciel d'exporter ses données dans son propre format. Pour PostgreSQL, `podman exec db pg_dump -U postgres > dump.sql` produit du SQL rejouable, **cohérent** même pendant que la base tourne, portable entre versions. C'est la sauvegarde de référence.
- **Sauvegarde physique** : on archive les fichiers du volume. `podman volume export pgdata -o pgdata.tar` (Podman), ou la méthode portable `podman run --rm -v pgdata:/data:ro -v ./:/backup alpine tar czf /backup/pgdata.tgz -C /data .`. Rapide et exacte, mais **le conteneur doit être arrêté** : copier à chaud les fichiers d'une base en cours d'écriture donne une archive incohérente, parfois irrécupérable.

8. En entreprise

- **La base de données** vit dans un volume nommé sur le poste de dev et en CI ; en production, elle est le plus souvent **hors conteneur** (service managé) ou sur un stockage réseau attaché au nœud par l'orchestrateur (PersistentVolume, labo 15). Jamais dans un bind mount sur le disque d'un serveur applicatif.
- **La configuration** (`application.yaml`, `nginx.conf`, certificats) arrive par bind mount en lecture seule — ou, au labo 08, par variables d'environnement et secrets.
- **Les logs** ne vont pas dans un fichier du conteneur : ils vont sur la sortie standard, et le moteur les collecte (labo 10). Un `logs/` monté en volume est un signe d'application mal adaptée.
- **Les uploads** d'utilisateurs (photos, PDF) vont dans un stockage objet (S3, MinIO), pas dans un volume local : un second réplica ne le verrait pas.

À retenir

- La couche writable meurt avec le conteneur ; une donnée qui doit survivre n'est pas *dans* le conteneur.
- Volume nommé pour les données du moteur, bind mount pour vos fichiers, tmpfs pour la mémoire.
- Une source qui commence par `/`, `./` ou `~` est un bind mount ; un simple nom est un volume — `./` oublié = volume vide.
- `VOLUME` dans une image crée des volumes anonymes : nommez les vôtres, supprimez avec `rm -v`.
- `:ro` par défaut ; `:Z` pour SELinux ; `:U` ou `--userns=keep-id` quand l'UID du conteneur ne possède pas le dossier.
- En rootless, l'UID 70 du conteneur est l'UID 100069 de l'hôte : `podman unshare` pour voir les fichiers.
- `pg_dump` pour une sauvegarde cohérente à chaud ; `volume export` ou `tar` seulement conteneur arrêté.

Vocabulaire

couche writable : couche overlay propre au conteneur, détruite avec lui. — **volume nommé** : répertoire géré par le moteur, persistant. — **volume anonyme** : volume créé par `VOLUME` sans nom. — **bind mount** : répertoire de l'hôte monté dans le conteneur. — **tmpfs** : système de fichiers en mémoire. — **copy-up** : copie du contenu de l'image dans un volume vide au premier montage. — **user namespace / subuid** : table de traduction des UID en rootless. — **`podman unshare`** : exécuter une commande dans le user namespace de Podman. — **sauvegarde logique / physique** : export par le logiciel / copie des fichiers.